

DOCUMENTAÇÃO TÉCNICA · SETEMBRO DE 2026

mecamecanica

Lakehouse de dados e IA para uma distribuidora de autopeças — do catálogo bruto à priorização de contatos comerciais com IA, de ponta a ponta.

Databricks Unity Catalog

Asset Bundles (DAB)

MLflow / scikit-learn

Genie (AI/BI)

Databricks Apps (AppKit)

Projeto pessoal de engenharia de dados e ciência de dados, construído sobre dados sintéticos de uma distribuidora fictícia de autopeças, com práticas de produção: testes de qualidade que derrubam o pipeline, auditoria de metadado, modelo registrado e versionado, e um ciclo fechado entre dado, modelo e ação humana.

1 Visão geral

O **mecamecanica** é um projeto de lakehouse completo — engenharia de dados, ciência de dados e um produto de dados (app + agentes de IA) — construído sobre o cenário de uma distribuidora de autopeças B2B fictícia, que atende oficinas mecânicas, autopeças independentes, concessionárias e redes de auto center em todo o Brasil.

O objetivo do projeto foi responder, com dado real e reproduzível, três perguntas que qualquer diretoria comercial faz:

- **Quem vai comprar?** — propensão de compra semanal, por cliente
- **Quem está sumindo?** — clientes em risco de churn
- **Quem eu ligo primeiro, e por quê?** — uma fila de contatos priorizada, com explicação em português

R\$ 102,3 mi

RECEITA, 24 MESES

~3.000

CLIENTES

42

VENDEDORES

16

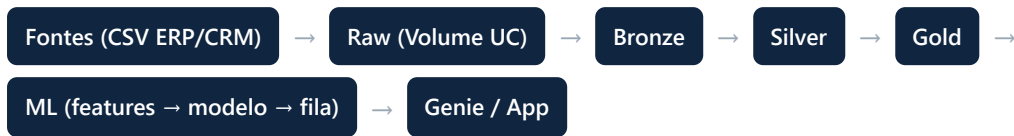
TAREFAS NO PIPELINE

Tudo é **código versionado**: catálogo, schemas, tabelas, testes, dashboard, modelo, agentes de IA e o app — nada foi criado clicando na interface do Databricks. O projeto roda inteiramente sobre o **Databricks Free Edition** (compute serverless), usando **Databricks Asset Bundles** como mecanismo único de deploy.

Por que isso importa: um catálogo de dados construído na interface não tem histórico, não tem revisão de código e não tem como ser recriado do zero de forma confiável. Este projeto trata infraestrutura de dados, modelo de ML e instruções de agente de IA exatamente como código de aplicação — com Git, commit e revisão.

2 Arquitetura: o padrão medalhão

O catálogo `lakehouse_mecamecanica` segue o padrão medalhão clássico, com uma camada de ciência de dados e uma de produto por cima da gold:



Camada	O que tem	Decisão de design
Raw	10 CSVs de ERP e CRM, exatamente como vieram, num Volume gerenciado do Unity Catalog	Uma tabela de controle confere que os 10 arquivos chegaram e não vieram vazios — a ausência de um arquivo não gera erro visível, gera número menor no dashboard
Bronze	10 tabelas Delta, colunas de negócio 100% <code>STRING</code>	Nenhuma limpeza aqui. Se o Spark inferir tipo, a sujeira proposital do dado (datas em dois formatos, por exemplo) já é perdida antes de virar evidência
Silver	10 tabelas limpas, tipadas, deduplicadas, com <code>CONSTRAINT</code> declarada	Deduplicação de clientes por CNPJ mantém o cadastro mais antigo e remapeia os <code>cliente_id</code> descartados — sem isso, pedidos antigos ficariam órfãos
Gold	4 dimensões, <code>fato_vendas</code> (grão: item de pedido), 3 marts, 6 views de negócio, 20 features de cliente, tabelas de modelo, a fila semanal	Devolução entra com sinal negativo, nunca é descartada — excluí-la infla a receita em ~R\$ 1,26 milhão

Qualidade de dados: duas redes de segurança

O pipeline tem **9 testes de qualidade** (conformidade de receita, unicidade de CNPJ, volume esperado do fato, integridade referencial) e uma **auditoria de metadado** — uma tarefa que quebra o job de propósito se qualquer tabela, view ou coluna da gold estiver sem `COMMENT`.

Por que a auditoria de metadado importa: o comentário de uma coluna não é documentação para humano ler — é o que o agente de IA (Genie) lê para decidir qual coluna usar numa pergunta em português. Uma coluna sem comentário é uma coluna que o agente vai errar, sem avisar ninguém. Essa auditoria encontrou e corrigiu um caso real neste projeto: uma coluna técnica (`_processado_em`) que nunca tinha sido comentada.

Sazonalidade invertida

O dado tem um comportamento real do setor de autopeças que qualquer análise ingênua lê ao contrário: o pico de vendas é o **mês anterior** à data de maior uso (abril, junho e outubro), e dezembro/janeiro são **vale esperado** — as oficinas já estão abastecidas, não é queda de desempenho. Essa regra está escrita como instrução explícita em todos os agentes de IA do projeto, para que nenhum deles confunda vale sazonal com problema.

3 O pipeline de produção

Um único job, `mecamecanica_pipeline`, orquestra as 16 tarefas, agendado diariamente às 6h (horário de Brasília):

```
raw_conferencia → bronze_ingestao → silver_clientes ┌
                                     │                 │ → gold_dimensoes → gold_fato_vendas → gold
                                     │                 │ → testes
                                     │                 │ → gold_metricas_negocio → auditoria
                                     │                 │ → gold_retorno_ligacao
                                     │                 │ → ml_features → ml_modelo → ml_fi
```

Cada camada usa a tecnologia certa para o problema: **Python/notebook** onde há laço e arquivo (ingestão, features, treino do modelo), **SQL** onde há transformação e teste. Nenhum wheel, nenhum build — o deploy inteiro do bundle leva segundos.

4 Ciência de dados: propensão de compra

As features: o que descreve um cliente

Vinte features, em quatro grupos (RFM, ritmo de compra, CRM, mix de produto), todas calculadas com uma disciplina só: **tudo que se sabia do cliente até uma data de corte** — nunca depois. A mesma função `montar_features()` gera a tabela de treino e a de score, o que torna estruturalmente impossível os dois divergirem.

A feature mais importante não veio de nenhuma biblioteca. `atraso_relativo` — a recência do cliente dividida pelo intervalo médio entre as próprias compras dele — nasceu de uma observação de negócio: dois clientes sumidos há 28 dias significam coisas opostas se um compra a cada 24 dias e o outro a cada 139. Ela sozinha bate as duas heurísticas que a intuição comercial sugere (recência crua e valor histórico), e acabou sendo a feature de maior peso no modelo.

O modelo, e a régua que prova que ele vale a pena

Antes de treinar qualquer coisa, o projeto mede três regras simples que uma gerência comercial já aplicaria de graça (ligar para quem comprou recentemente, para quem compra mais, para quem está atrasado) — essa é a régua que o modelo precisa vencer para justificar existir.

10,1% TAXA-BASE (LIGAR ÀS CEGAS)	0,85 AUC DO MODELO	4,2× LIFT NOS 200 PRIMEIROS	85/200 ACERTOS ESPERADOS
---	------------------------------	---------------------------------------	------------------------------------

O algoritmo é `HistGradientBoostingClassifier` (scikit-learn) — trata valores nulos nativamente, o que importa porque as features de ritmo são nulas de propósito para clientes de um pedido só. O modelo é registrado no **Unity Catalog** com o alias `@prod`, versionado a cada novo treino, com três testes automáticos que **interrompem o job** se o modelo não bater a melhor regra simples por margem suficiente, se o AUC vier bom demais (sinal de vazamento de dado) ou se o lift ficar baixo demais para justificar a operação.

A fila semanal e o agente

O modelo pontua todos os clientes; a fila seleciona os **200 contatos da semana**, distribuídos pela carteira de cada vendedor, com uma frase em português explicando o motivo e uma sugestão de recompra.

Duas melhorias implementadas além do que qualquer roteiro pedia:

- **Priorização por valor esperado** (score × margem × ticket médio), não só por probabilidade de compra — medido: troca 29 dos 200 contatos e aumenta o valor esperado da fila em **+3,1%**, com o mesmo esforço de ligação.

- **Sugestão nunca oferece produto sem estoque** — antes, 17% das sugestões (34 de 200) ofereciam saldo zero. Agora a fila verifica disponibilidade antes de sugerir, com um plano B (marca alternativa) quando todo o histórico do cliente está em ruptura.

5 Agentes de IA: um Genie por audiência

O projeto tem **três espaços Genie** (AI/BI), deliberadamente separados por quem pergunta — a mesma lição vale para agentes quanto vale para APIs: instruções de audiências diferentes brigam entre si se moram no mesmo lugar.

Genie	Audiência	Fontes	Regra central
Comercial	Qualquer pergunta de negócio	12 (gold inteira)	Sazonalidade invertida; nunca usar a bronze
Direção	Decisão executiva sobre a fila	7 (fila, score, métricas do modelo, retorno)	Métrica é <code>lift_top200</code> — nunca cite AUC para responder pergunta de negócio
Fila da semana	O vendedor, na ligação	Fila + 4 funções SQL	Funções: <code>priorizar_carteira</code> , <code>contexto_cliente</code> , <code>sugerir_produtos</code> , <code>checar_disponibilidade</code>

Todas as respostas trazem o SQL gerado — o hábito que separa quem *usa* Genie de quem *confia* em Genie sem conferir.

6 O produto: o app que fecha o ciclo

Um Databricks App (AppKit — Node/TypeScript/React) com três telas, lendo e **escrevendo** de volta na gold — o único dos três "portais" de consumo (dashboard, Genie, app) que faz isso:

Tela	O quê
A semana	4 KPIs (contatos, receita esperada, conversão prevista, já trabalhados), fila filtrável por vendedor, e os 4 botões de retorno de ligação (Vendeu / Vai pensar / Sem interesse / Não atendeu)
Acompanhamento	Cobertura da fila, conversão real × prevista pelo modelo, desfecho por vendedor
Perguntar	O Genie da Direção embutido no produto

O ciclo fechado: pipeline → score → fila → ligação → retorno do vendedor → `gold.retorno_ligacao`.
Testado ponta a ponta: um clique real gravou uma linha com o e-mail de quem clicou, o KPI "já trabalhados" mudou na hora, e o Genie — sem nenhuma linha de código alterada — passou a responder com o dado novo.

A escrita usa uma rota única (`POST /api/retorno`), validada por um contrato fechado de 4 valores antes de tocar no banco — um corpo inválido nunca chega ao warehouse. O app roda como um usuário próprio do Unity Catalog (service principal), com permissão de escrita concedida numa **tabela só**, nunca no schema inteiro.

7 Um incidente real, e o que ele ensinou

No meio do projeto, o antivírus corporativo da máquina encerrou uma sessão de trabalho e apagou código-fonte que nunca tinha sido commitado. Os dados, o modelo registrado e os agentes de IA sobreviveram no workspace — só o código-fonte se perdeu.

A recuperação não foi reescrever tudo de memória: foi **engenharia reversa do próprio catálogo** — schemas e comentários exatos via `DESCRIBE TABLE EXTENDED`, corpo de função via `DESCRIBE FUNCTION EXTENDED`, definição de view via `SHOW CREATE TABLE`, e o Genie Space completo via `databricks bundle generate genie-space`. Onde a lógica não podia ser recuperada sem perda (a lógica de treino do modelo, por exemplo), o próprio material do curso original foi localizado e comparado linha a linha com a reconstrução — processo que revelou bugs reais que teriam quebrado a próxima execução agendada.

O resultado: depois da correção, o modelo retreinado bateu *exatamente* as métricas históricas já registradas — confirmando que a correção reproduziu a metodologia original, não uma aproximação.

8 Ideias registradas para o futuro

Duas ideias foram avaliadas e conscientemente adiadas, por dependerem de volume de dado que ainda não existe:

- **Fila de win-back** — hoje `gold.clientes_em_risco` (clientes há mais de 90 dias sem comprar) só existe como métrica de consulta. Poderia virar uma segunda fila acionável, paralela à de propensão de compra.
- **retorno_ligacao como feature do modelo** — o desfecho de uma ligação anterior (em especial uma rejeição explícita) é um sinal que nenhuma feature atual capta. Ainda não há histórico suficiente acumulado para justificar o esforço, e a coleta é enviesada (só cobre quem a própria fila já escolhe chamar).

mecamecanica — projeto pessoal de engenharia de dados, ciência de dados e agentes de IA sobre Databricks. Dados sintéticos, arquitetura e práticas de produção real. Documento gerado em setembro de 2026.